trvale-book — Main Project Report

trvale-book: An AI-Powered Travel Planner

Abstract

trvale-book is a browser-based travel-planning application that creates a personalized, budget-aware itinerary from a traveller's destination, dates, group, budget, travel style, pace, interests and dietary needs. A five-step wizard collects the inputs. A FastAPI backend validates them, requests a structured plan from Google Gemini or OpenAI (or uses a built-in fallback generator when no key is configured) and calculates a budget breakdown. The frontend lets users review the plan by day, edit or regenerate activities, save and search trips, and print or save the itinerary as a PDF.

The project separates concerns. A React 19, Vite and Tailwind CSS frontend, deployed on Vercel, handles input and presentation. A Python FastAPI backend, prepared for Render, owns validation, AI orchestration, budget logic and trip operations. Supabase provides authentication and a PostgreSQL database protected by row-level security. Secret keys stay on the server, each layer can be tested on its own, and a zero-setup demo mode lets anyone explore the whole application.

The application is positioned as a planning assistant. It does not book transport or accommodation and does not guarantee prices or availability. Its value is removing the blank-page problem: the traveller receives a structured day-by-day starting point that can be corrected and saved.

Project at a glance

Item	Details
Project name	trvale-book
Frontend	React 19, Vite 8, Tailwind CSS 3.4, React Router 7; hosted on Vercel
Backend	Python 3.11 and FastAPI; Render configuration included
AI service	Google Gemini (gemini-1.5-flash by default) or OpenAI (gpt-4o-mini), with heuristic fallback
Database and auth	Supabase (PostgreSQL, Auth, row-level security)
Key features	5-step wizard, AI itinerary, live budget summary, editing, saved trips, print / save as PDF, demo mode
Live frontend	https://travel-book-livid.vercel.app

How this report is organised

Section	Content
1-2	Project description, objectives, scope and usage scenarios
3-4	Technical architecture, prerequisites and configuration
5-6	Project workflow and backend API overview
7	Website pages with screenshots
8-10	Testing plan, limitations, security, future scope, deployment and conclusion

1. Project Description

The purpose of trvale-book is to simplify the first stage of trip planning. Instead of starting from a blank page, a traveller submits preferences and receives a day-wise itinerary to review, adjust and keep. The application is a planning assistant, not a booking or travel-provider platform.

Problem addressed

Planning usually means coordinating destination, dates, group size, budget, pace, activities and food needs across many sources. A traveller opens several websites, copies details into notes and spreadsheets, and rebalances the plan whenever a constraint changes. The project combines these inputs in one interface and produces an itinerary draft with a running cost estimate.

Project objectives

- Collect essential trip preferences through a guided five-step wizard with validation.
- Generate a day-by-day plan with morning, afternoon and evening activities, estimated costs and tips.
- Show a live budget comparison and warn when the plan exceeds the target budget.
- Allow users to edit activities, regenerate a day, save, search, filter and delete trips.
- Provide secure authentication and per-user data access through Supabase, plus a one-click demo mode.
- Support browser printing or Save as PDF so the plan can be taken offline.

Target users

Independent travellers, students, couples, families and small groups who want a quick personalised starting plan rather than a fully managed tour. Because the inputs are common planning details, the application is approachable for first-time users and does not require an account in demo mode.

Scope of the project

In scope	Out of scope (current version)
Five-step wizard with input validation	Booking of flights, trains, hotels or activities
AI-generated structured itinerary with estimated costs	Guaranteed or live prices and availability
Live budget summary and warnings	Ratings, reviews and opening hours retrieved by the app
Saving, searching, editing and deleting trips	Complete database persistence of days and activities on every route
Supabase authentication and row-level security	Server-side PDF generation (print uses the browser)
Print / Save as PDF and printable HTML export	Model-backed regeneration of a single day

Design principles

- Simplicity first: one wizard, one itinerary view and one saved-trips list.
- Structured output: the AI reply is parsed and normalized before it is stored or shown.
- Graceful degradation: fallback generators keep the flow working when a service is unavailable.
- Transparency: costs are labelled as estimates and a banner shows when demo mode is active.
- Security by separation: API keys and database credentials stay on the backend.

2. Project Description and Scenarios

The scenarios below describe how a traveller is expected to use the application from first visit to final output. Each scenario maps to pages and endpoints described later in this report.

Scenario 1: Generate a personalized itinerary

A traveller signs in (or uses the demo sign-in), starts a new plan and enters Kyoto, Japan as the destination, 10 to 13 November 2026 as the dates, two travellers as a couple, a budget of 3,200 USD, a balanced style, a moderate pace, and the interests Culture and Heritage, Food and Culinary and Sightseeing, with the dietary note "Pescatarian, loves matcha and ramen". After the final review the app displays a four-day itinerary with a budget summary. The figures are planning estimates.

Outcome: a structured itinerary with a title, day-wise activities in time slots, an accommodation suggestion and transit tip per day, and a cost breakdown.

Scenario 2: Adjust the plan and watch the budget

The traveller opens the edit page, changes an activity, deletes another and regenerates a day. The budget summary recalculates and shows a warning when the estimate exceeds the target budget.

Outcome: an edited plan whose cost estimate matches the changes.

Scenario 3: Save and revisit a trip

Trips appear on the dashboard and on Saved Trips, where they can be searched by title or destination and filtered by travel style. A trip can be opened, edited or deleted after a confirmation dialog.

Outcome: a persistent collection of plans that can be reopened without generating them again.

Scenario 4: Print or save the plan

The traveller chooses Print / PDF on the itinerary page. A print stylesheet hides navigation, sidebar, tabs and buttons and prints every day. The user selects a printer or Save as PDF.

Outcome: a portable copy of the itinerary for co-travellers or offline use.

Scenario summary

Scenario	Trigger	Main components	Result
1. Generate itinerary	Wizard submitted	Frontend, FastAPI, AI provider or fallback	Itinerary and budget shown
2. Adjust the plan	Edit page actions	Frontend, FastAPI, budget service	Updated plan and budget
3. Save and revisit	Open Saved Trips	Frontend, FastAPI, Supabase or demo store	Trip reopened, edited or deleted
4. Print or save	Print / PDF chosen	Browser print dialog	Printed copy or PDF

3. Technical Architecture

The application uses a separate frontend and backend. The React frontend runs on Vercel and sends JSON requests to the FastAPI backend. The backend calls the AI provider for generation and Supabase for authentication and storage. If the backend is unreachable, the frontend falls back to a browser-side generator and a localStorage demo store.

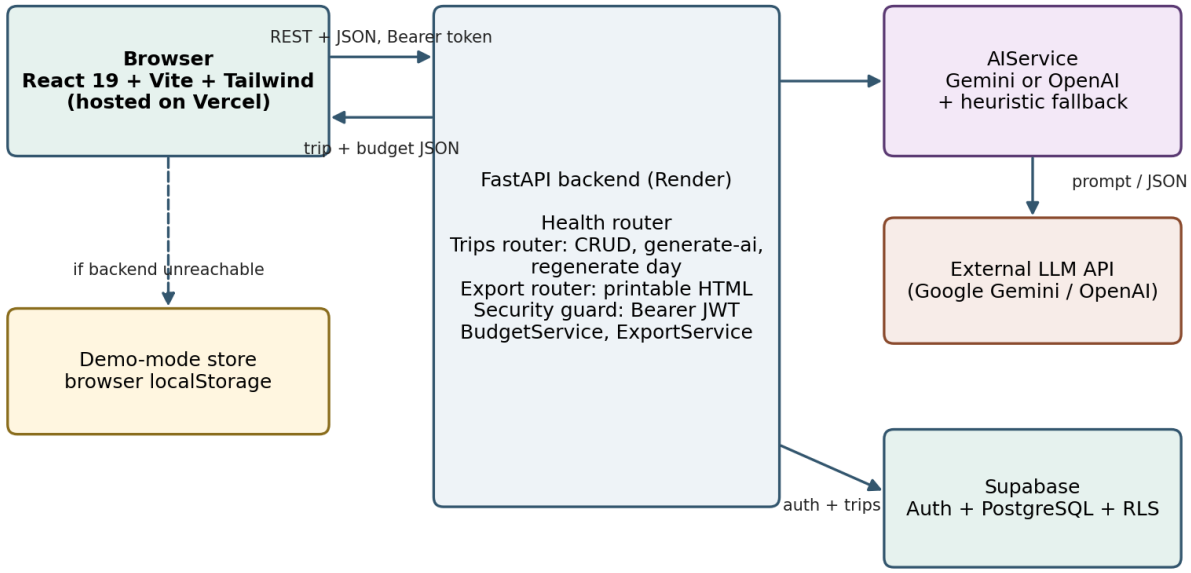


Figure 1. High-level architecture of trvale-book (secret keys stay on the backend).

Figure 1. High-level architecture of trvale-book.

How the layers interact

The traveller interacts only with the browser. The frontend converts the wizard values into a JSON request and calls the backend over HTTP with a bearer token. The backend validates the request, builds a prompt, calls the model, normalizes the reply, computes the budget and returns the trip. Trip operations are authorized through the Supabase token, and row-level security protects the database.

Component	Technology	Responsibility
Frontend	React 19, Vite 8, Tailwind CSS, React Router 7	Pages, components, routing, state, print styles
API client	Axios with bearer token and local fallbacks	Calls the backend; falls back to demo data
Backend	Python 3.11, FastAPI, Uvicorn	Routing, validation, orchestration
AI service	httpx; Gemini or OpenAI; heuristic generator	Itinerary generation and day regeneration
Budget and export	BudgetService, ExportService	Cost breakdown, warnings, printable HTML
Database and auth	Supabase PostgreSQL, Auth, RLS	Profiles, trips, days, activities, ownership
Hosting	Vercel and Render	Deploy frontend and backend separately

Why this architecture

- Security: provider keys and service credentials never reach the browser.
- Independent deployment: the frontend and backend can be redeployed separately.
- Replaceable parts: the AI provider or the database can be changed by editing one backend module.
- Resilience: fallbacks keep the demonstration usable when a service is down.

4. Prerequisites and Configuration

Prerequisite	Purpose
Node.js and npm	Install dependencies, run, test and build the React/Vite frontend
Python 3.11 and pip	Create the backend environment and install FastAPI, Pydantic, httpx and the Supabase client
Gemini or OpenAI API key (private backend variable)	Authenticate calls for live itinerary generation
Supabase project	Authentication and storage; apply supabase/migrations/001_initial_schema.sql
Git and a modern browser	Clone the repository, run the app and view results
Vercel and Render accounts	Deploy the frontend and backend

Configuration values

Secrets such as the AI key, the Supabase key and the JWT secret must stay in environment configuration and must not appear in documentation, screenshots or source control. The supplied source archive contains only .env.example files with placeholder values.

Setting	Where it is set	Notes
GEMINI_API_KEY or OPENAI_API_KEY	Backend environment	Server-side only; selected by AI_PROVIDER
SUPABASE_URL, SUPABASE_KEY, SUPABASE_JWT_SECRET	Backend environment	Token verification and database access
CORS_ORIGINS	Backend environment	Must include https://travel-book-livid.vercel.app
ENABLE_DEMO_MODE	Backend environment	Set to false for production with real users
VITE_API_BASE_URL	Frontend (Vercel)	Points the frontend to the backend
VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY	Frontend (Vercel)	Enable live authentication; omit for demo mode

Local setup outline

1. Clone the repository and open the project folder.
2. In frontend, run npm install and npm run dev; open http://localhost:5173.
3. In backend, create a virtual environment, run pip install -r requirements.txt and provide private variables.
4. Start the API with uvicorn app.main:app --reload --port 8000 and open /health.
5. Generate a test itinerary from the wizard.

Deployment checklist

- Frontend deployed on Vercel with vercel.json for client-side routes.
- Backend deployed on Render with private variables entered in its dashboard.
- VITE_API_BASE_URL pointing to the backend; CORS_ORIGINS listing the Vercel origin.
- The /health endpoint checked after every deployment.

5. Project Workflow

Step	Action	Purpose
1	The user completes the five wizard steps in the frontend.	Capture the preferences that shape the plan
2	The frontend posts the request to /api/trips/generate-ai.	Hand the request to the backend
3	Pydantic validates the request.	Reject malformed data early
4	The AI service builds a prompt and calls Gemini or OpenAI, or uses the fallback generator.	Obtain a draft itinerary
5	The reply is parsed and hydrated with identifiers and defaults.	Ensure the output is usable
6	The budget service computes the breakdown and warnings.	Compare the plan with the target budget
7	The frontend displays the itinerary; the user edits, saves, deletes or prints.	Present and manage the result

6. Backend API Overview

The backend exposes a small REST interface. All request and response bodies are JSON except the HTML export. Trips routes require a bearer token; in demo mode a demo token is accepted.

Method	Endpoint	Purpose
GET	/health	Status, version, environment, Supabase and AI configuration flags
GET	/	Service name, version and links to /docs and /health
GET	/api/trips	List the user's trips with budget breakdown
POST	/api/trips	Create and save a trip
GET	/api/trips/{trip_id}	Retrieve one trip (ownership checked)
PATCH	/api/trips/{trip_id}	Update trip metadata
DELETE	/api/trips/{trip_id}	Delete a trip
POST	/api/trips/generate-ai	Generate an itinerary from wizard preferences
POST	/api/trips/{trip_id}/days/{day_number}/regenerate	Refresh the activities of one day
POST	/api/trips/{trip_id}/export	Return standalone printable HTML for a trip

Notes on the API

- Only the generate endpoint calls an AI provider; all other endpoints read, change or format data.
- Request validation rejects an end date before the start date, a budget below 50, and traveller counts outside 1 to 50.
- Unknown trips return 404 and trips owned by another user return 403.
- Interactive API documentation is available at /docs when the backend runs.

Database summary

The migration creates five tables with row-level security: profiles, trips, trip_preferences, itinerary_days and itinerary_activities. Updated-at triggers keep timestamps current, and a signup trigger creates a profile for each new user.

7. Exploring the Website's Web Pages

The pages below are from the deployed frontend at https://travel-book-livid.vercel.app. Each figure is a screenshot of the corresponding page.

7.1 Landing and sign-in

The landing page introduces the product with a hero section, a sample itinerary preview and the three-step explanation. The sign-in page offers email and password sign-in and a one-click demo sign-in.

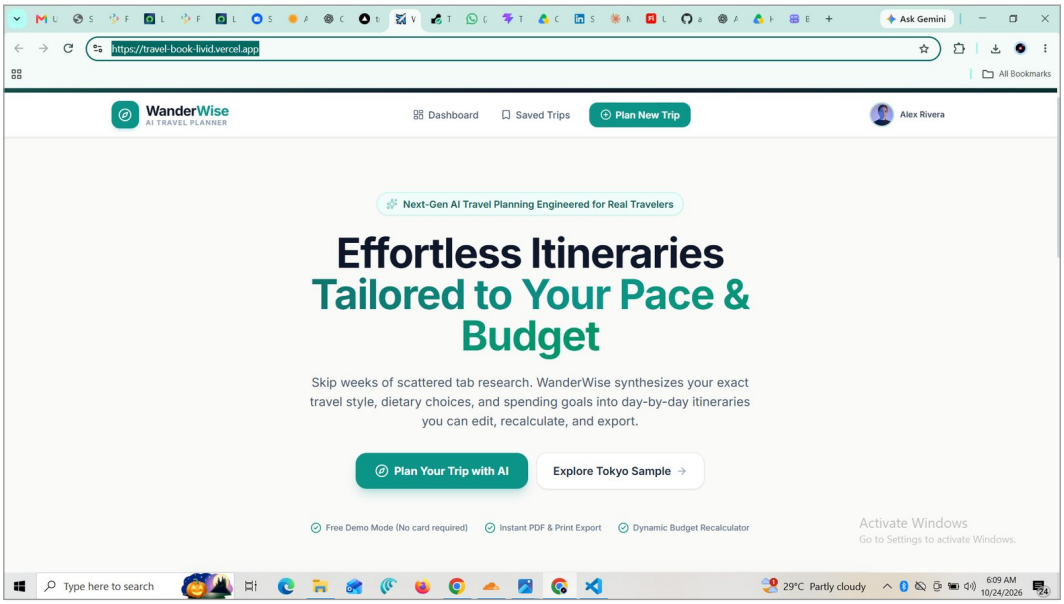


Figure 2. Landing page.

7.2 Dashboard

After sign-in the dashboard shows trip statistics, the user's active itineraries and curated inspiration destinations, with a button to start a new plan.

7.3 Trip wizard

The Trip Architect wizard has five steps: Destination and Dates; Travelers and Budget; Style and Interests; Dietary and Requirements; and Final Review and Synthesize. Each step is validated before the user can continue.

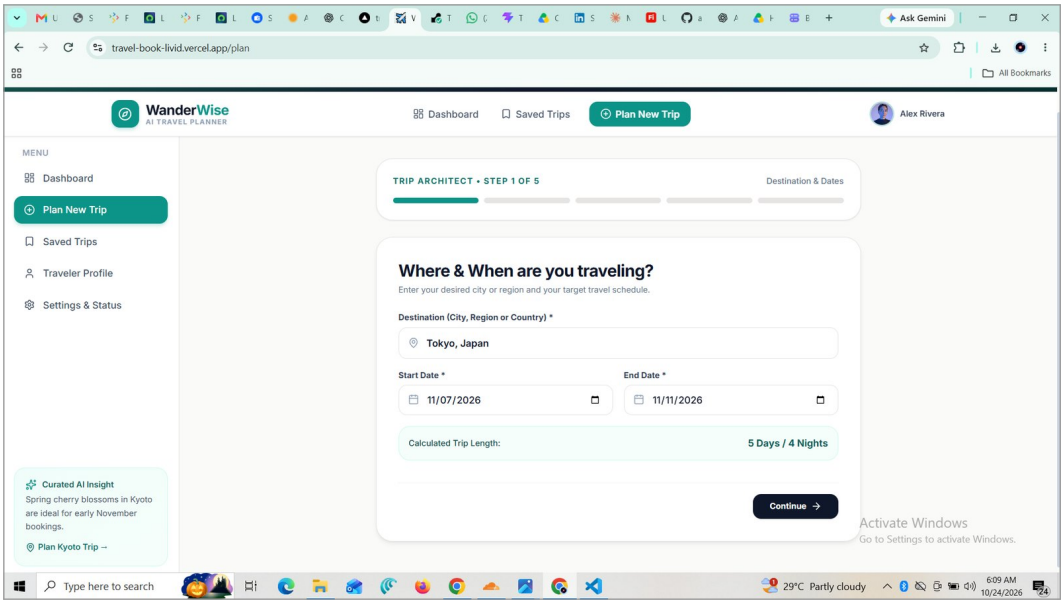


Figure 3. Trip wizard, step 1: destination and dates.

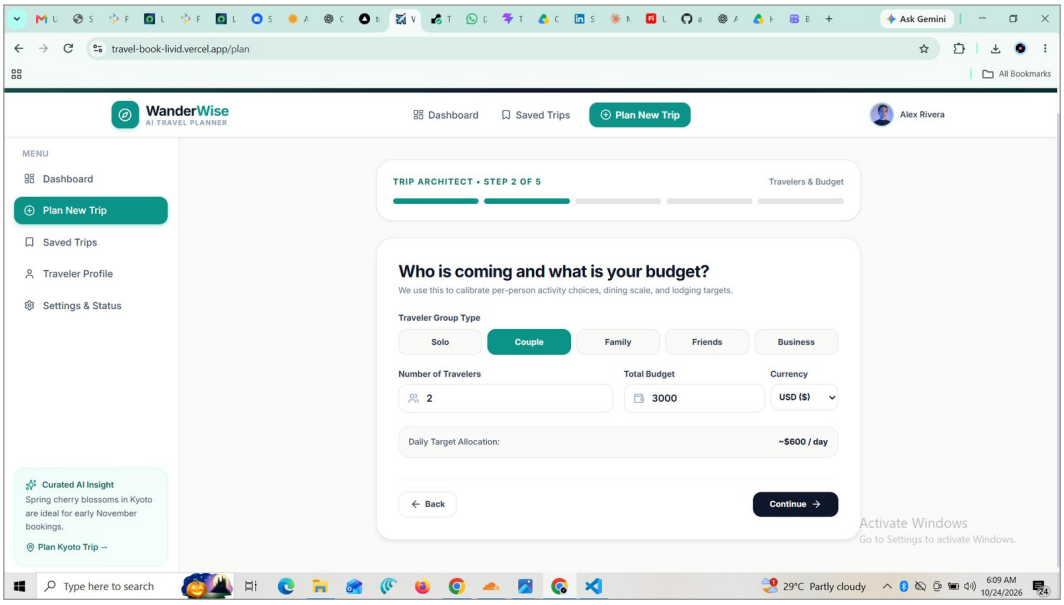


Figure 4. Trip wizard, step 2: travelers and budget, with the daily target allocation.

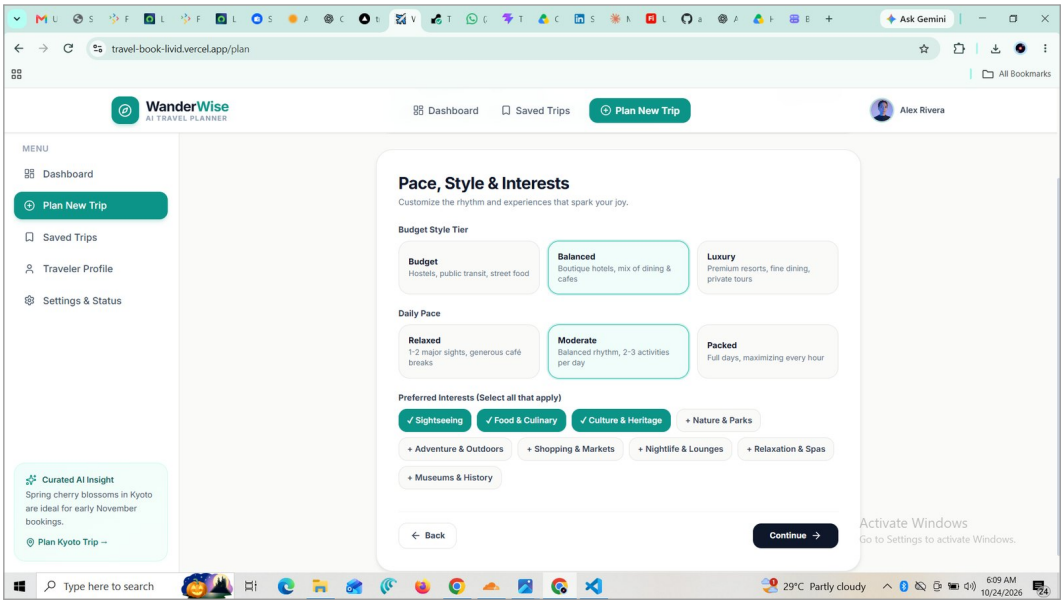


Figure 5. Trip wizard, step 3: budget style, daily pace and interests.

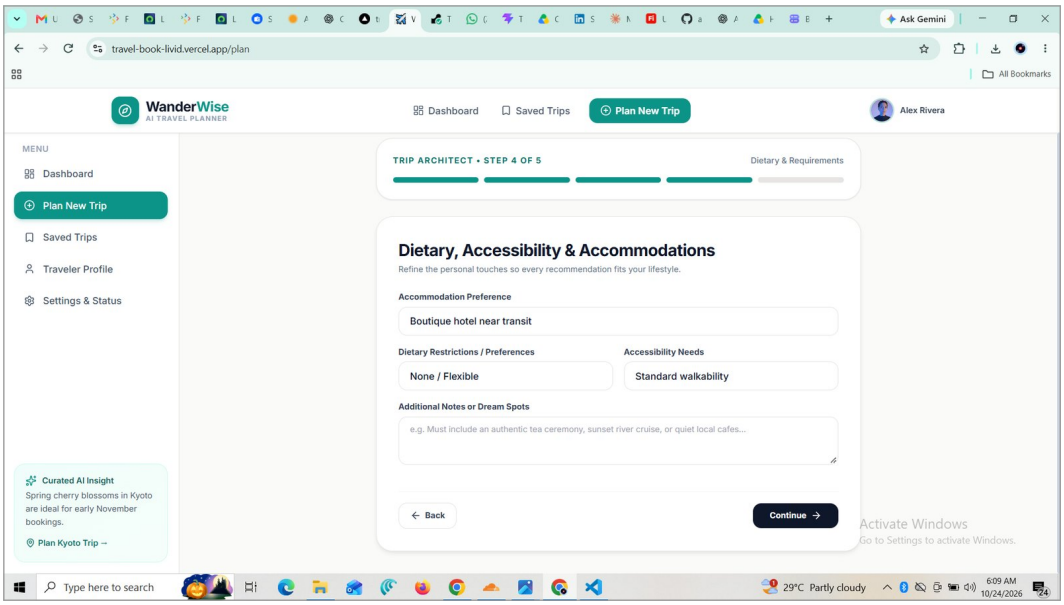


Figure 6. Trip wizard, step 4: dietary, accessibility and accommodation requirements.

Wizard field	Example	Use in the itinerary
Destination	Kyoto, Japan	Sets the place context
Start and end date	2026-11-10 to 2026-11-13	Determines the number of days to plan
Traveller type and count	Couple, 2	Scales costs and tone
Budget and currency	3,200 USD	Guides estimated costs and the budget summary
Style and pace	Balanced, moderate	Sets accommodation level and activities per day
Interests	Culture, Food, Sightseeing	Shapes the type of activities
Dietary and accessibility	Pescatarian; subway elevator access	Passed to the AI prompt

7.4 Generated itinerary and budget

The itinerary page presents the trip header, a budget summary and day tabs. Each activity shows its time slot, start time, duration, estimated cost, location and a booking tip. The Print / PDF button opens the browser print dialog.

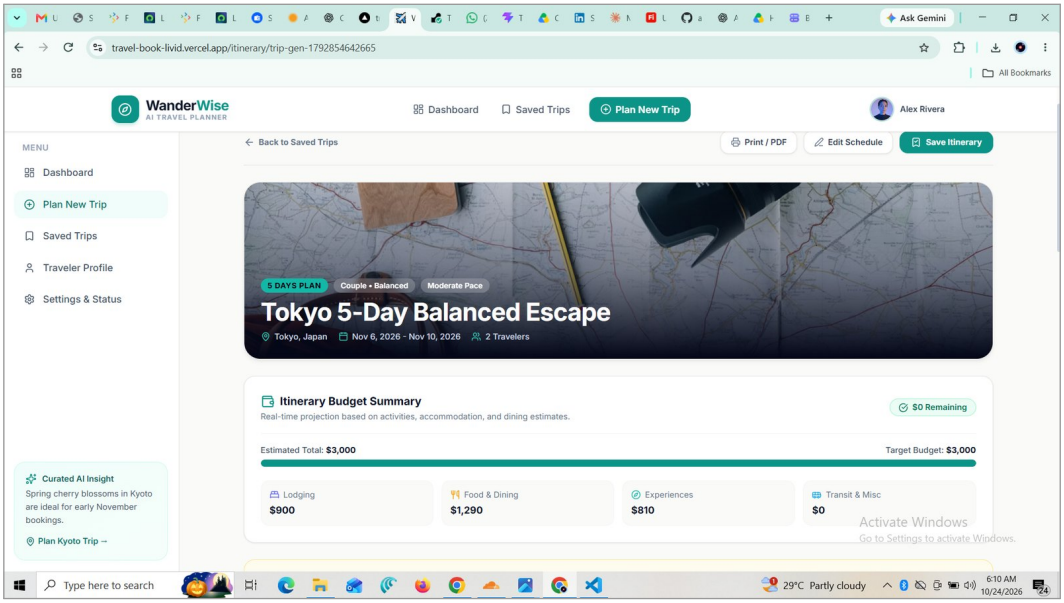


Figure 7. Generated itinerary with trip header and budget summary.

7.5 Saved Trips and editing

Saved Trips lists plans with a search box and a travel-style filter. The edit page allows activities to be changed or deleted and a day to be regenerated.

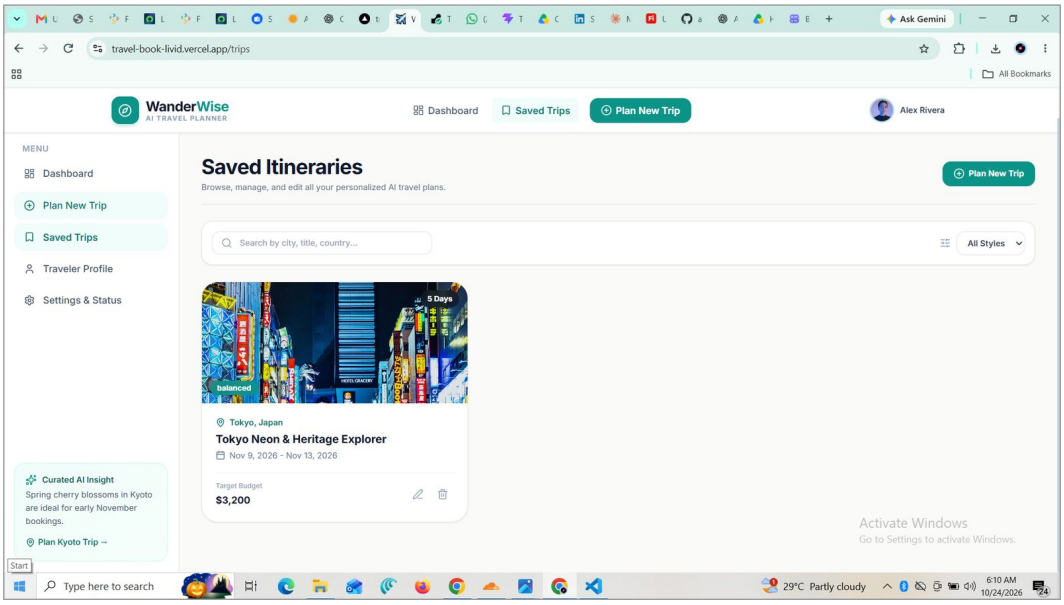


Figure 8. Saved Trips with search and style filter.

7.6 Profile, settings and 404

The profile page stores default currency, travel style, pace and dietary and accessibility notes. The settings page shows system diagnostics and lets the user reset demo data. Unknown addresses show a custom 404 page.

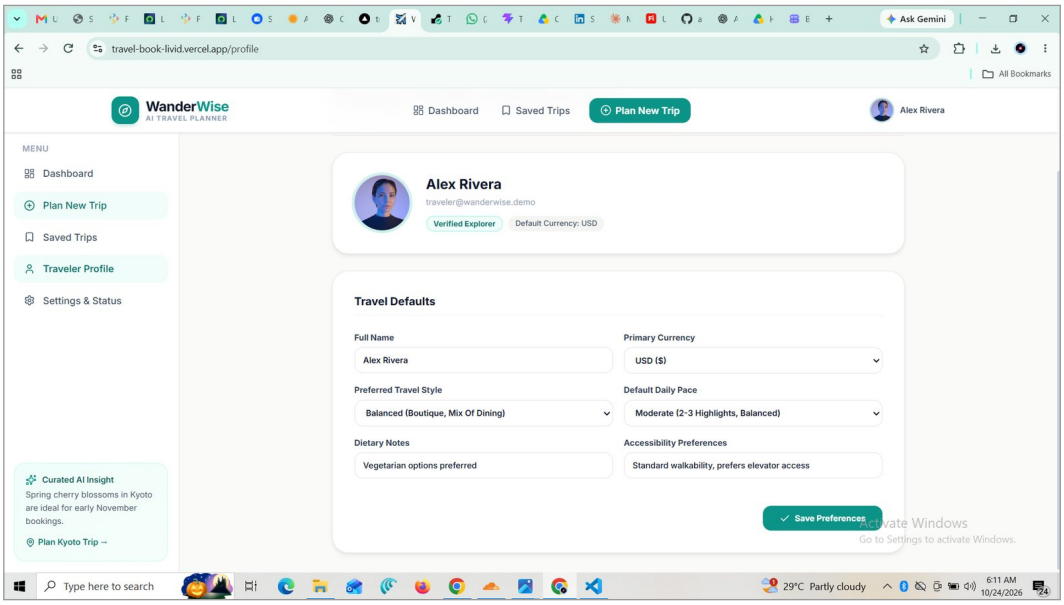


Figure 9. Traveler profile and default preferences.

8. Testing and Validation Plan

The table lists checks to run against the deployed application. Each row states the behaviour that counts as a pass. The repository also contains 17 backend pytest tests and 6 frontend Vitest tests; their results should be reproduced and recorded.

Test	Expected behaviour	Status
Frontend loading	Landing page renders without console or build errors	Pending verification
Demo sign-in	One-click sign-in opens the dashboard with the demo banner	Pending verification
Wizard validation	Blank destination, reversed dates, low budget and no interest are blocked	Pending verification
Itinerary generation	Valid input produces day tabs, activities and a budget summary	Pending verification
Budget warning	An over-budget plan shows a warning	Pending verification
Edit and regenerate	Edits change the plan and total cost	Pending verification
Saved trips	Search, filter, open and delete behave correctly	Pending verification
Print / save	Print preview shows all days without navigation or buttons	Pending verification
Backend health	/health responds if a backend is deployed	Pending verification
Automated tests	pytest and npm run test pass	Pending verification

These are planned checks, not claimed results. No latency, throughput or success-rate values should be reported unless measured.

Suggested evidence

- Screenshots of the generated itinerary and of Saved Trips after a reload.
- A print preview of a saved itinerary.
- The terminal output of pytest and npm run test.
- A short note of any failed test, its cause and the fix applied.

9. Limitations, Security and Future Scope

Current limitations

- AI-generated suggestions and costs are estimates, not quotations.
- The backend currently keeps generated and demo trips in an in-memory store and inserts only the trip row into Supabase on creation; days and activities are not yet persisted by every route.
- Day regeneration produces alternative entries by transforming existing activities; it is not a new model call.
- The AI retry setting is declared but not yet applied in provider calls.
- Availability of the AI provider and the database depends on configuration, quotas and connectivity.

Security considerations

- Keep API keys and database credentials on the server side.
- Row-level security restricts each table to its owner.
- Disable demo mode and restrict CORS for production.
- Avoid exposing tokens or environment variables in screenshots and logs.

Future enhancements

- Complete Supabase persistence for all trip routes and automated end-to-end tests.
- Model-backed day regeneration and clear labelling of fallback output.

- Approved live place-data and weather integrations.
- Authorized booking integrations.
- Accessibility, multilingual support, rate limiting and monitoring.

10. Deployment and References

Resource	Address
Live frontend (Vercel)	https://travel-book-livid.vercel.app
Backend and health endpoint	Not supplied - add the deployed backend URL and /health
Source repository	Not supplied - add the GitHub link
Demonstration video	To be added after recording

Implementation references: the uploaded project source archive; frontend source under frontend/src; backend routes in backend/app/api; AI processing in backend/app/services/ai_service.py; budget logic in budget_service.py; models in backend/app/models/schemas.py; and the database migration in supabase/migrations.

Conclusion

trvale-book brings together preference-based itinerary generation, a live budget summary, editing, saved trips and browser-based printing. It separates the frontend, API, AI service and database responsibilities, which keeps the design easy to explain, test and extend. Before final submission, the testing checklist should be completed with observed results, screenshots should be inserted in Section 7 and the demonstration video link should be added. With complete database persistence, live place data and automated tests, the application can grow from a planning assistant into a fuller travel-support tool.

Pre-submission checklist

- All tests in Section 8 executed and their status updated with observed outcomes.
- All eleven screenshots inserted in Section 7.
- Backend and repository links added to Section 10.
- The demonstration video link added.
- No API keys, tokens or .env contents visible in any screenshot or log.